

Broadcast, Multicast and Discovery

COMP2014 Distributed Systems

Chris Greenhalgh

School of Computer Science

Contents

- IP Broadcast & Multicast
- Java Multicast API

(examples:)

- Discovery: DHCP, SSDP/UPnP, mDNS, DNS Service Discovery
- LAN gaming

IP Multicast and broadcast

IP Broadcast (1)

- IP **Broadcast** allows a single **UDP datagram** to be sent to **ALL** of the hosts on a physical network
 - Datagram is addressed to an IP **broadcast address**
 - A **broadcast** address effectively represents **ALL** hosts on a network
- Address **255.255.255.255** = broadcast address for “this” network
 - the highest IP address on a subnet is the broadcast address for that subnet
 - E.g. subnet 128.243.21.0/24 (netmask 255.255.255.0) has broadcast address 128.243.21.255
 - Will be forwarded unicast to the right network (may be filtered by firewall)

IP Broadcast (2)

- On the destination network, the IP broadcast address is mapped to a **link layer** broadcast address
 - E.g. Ethernet or WiFi broadcast MAC address
- The **physical network** copies the broadcast frame to all hosts
 - Either because it is a physical shared medium (old shared-medium Ethernet), or because the switch copies the frame to all hosts (Ethernet over UTP) or the AP resends it to all hosts (WiFi)
- All physical **network interfaces** accept frames addressed to the broadcast MAC address
 - Then the frame (IP datagram) is passed to the OS to handle

IP Multicast (1)

- (like broadcast, but) IP **Multicast** allows a single **UDP datagram** to be sent efficiently to many (but not all) hosts
 - (Note, usually on the same network – see later)
- Any host can choose to **join** any number of groups
 - OS then configures **network card** to accept corresponding frames,
 - and informs local router(s)/switches via Internet Group Management Protocol (**IGMP**)

Q: why does the network card need to know?

IP Multicast (2)

- Datagram is addressed to an IP **multicast address**
 - This is mapped to a **link layer** multicast address
 - E.g. Ethernet or WiFi MAC address
- The **physical network** copies the frame to all relevant hosts
 - Either because it is a physical shared medium, or because the switch or AP copies the frame to all (relevant) hosts
 - the switch or AP may monitor IGMP packets to determine group membership and only send copies of the frame to interested hosts
- The physical **network interface** accepts the multicast frame
 - If and only if the OS has explicitly **joined** that group already

IP Multicast Addresses

- A **multicast address** is a Class D IPv4 address
 - The top 4 bits are 1110,
i.e. 224.0.0.0 to 239.255.255.255
 - These addresses represent **groups** of machines rather than individual hosts (or networks)
- Specific (ranges of) multicast addresses are assigned to specific purposes, e.g.
 - Local Network Control Block (224.0.0.0 to 224.0.0.225), for multicast traffic within a given local network.
 - Administratively Scoped Block (239.0.0.0 to 239.255.255.255), for multicast traffic within a particular administrative region (e.g. an organisation's network)
 - For use by specific protocols, managed by IANA
 - <https://www.iana.org/assignments/multicast-addresses/multicast-addresses.xhtml>

Java Multicast API

(java.net)

- Extends UDP API inc. **DatagramPacket**
- **MulticastSocket** – extends **DatagramSocket**
 - Constructor specifies explicit **port** to bind to, else any free
 - **joinGroup(group)** and **leaveGroup(group)** indicate that socket/host should join/leave specified group
 - **time to live** limits maximum network hops
 - default is 1, i.e. local network only; unicast is normally 255; 0 is loopback only
 - **Loopback**, i.e. whether messages are also delivered on the sending machine
 - **ReuseAddress**, i.e. whether another process can bind to the same port on the same host to receive copies of the same messages

Multicast server: joins a group and waits for simple (multicast) UDP requests and responds (unicast)

```
import java.net.*;
import java.io.*;
public class MulticastServer{
    public static void main(String args[]){
        // args give multicast group (e.g. "228.5.6.7")
        MulticastSocket s =null;
        try {
            InetAddress group = InetAddress.getByName(args[0]);
            aSocket = new MulticastSocket(6788);
            aSocket.setReuseAddress(true); // ...
            aSocket.joinGroup(group);
            byte[] buffer = new byte[1000];
            while(true){
                DatagramPacket request = new DatagramPacket(buffer, buffer.length);
                aSocket.receive(request);

                // this figure continued on the next slide
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

1. **(Determine multicast group)**
2. **Make a MulticastSocket**
3. **Allow other sockets to bind same port**
4. **.joinGroup the multicast group**
5. **Make an empty DatagramPacket**
6. **.receive a DatagramPacket as a (multicast) UDP packet**

Multicast server, *continued*

1. **Make an response DatagramPacket**
2. **.send it (unicast in this case) back to the client**
3. **Loop back...**
4. **.close (or .leaveGroup) to stop receiving**

```
// this bit is just a unicast reply, but multicast only differs in dest. address
DatagramPacket reply = new DatagramPacket(request.getData(),
      request.getLength(), request.getAddress(), request.getPort());
aSocket.send(reply);
}
}catch (SocketException e){System.out.println("Socket: " + e.getMessage());
}catch (IOException e) {System.out.println("IO: " + e.getMessage());
}finally {if(aSocket != null) aSocket.close();}
}
}
```

Multicast considerations (1)

- **Failure model** – same as UDP, i.e. best effort
 - Any *subset* of hosts may receive a given message
 - Messages may be re-ordered across internetworks
- Making multicast **reliable** is complicated, e.g.
 - Are acknowledgements unicast or multicast?
 - Does every receiver send an acknowledgement?
 - => acknowledgements flood the network or overload the sender
 - Are retransmissions multicast or unicast?
- What about **flow control** and **congestion control**?
 - What do you do if just one receiver can't keep up?
 - => throw them out? Or make everyone “suffer” with them? (see also next slide)

Multicast considerations (2)

- **Wide-area support** – Multicast works well on Ethernet and Wifi but is often NOT forwarded by routers
 - E.g. what if a host joins a very busy multicast group?
 - => lots of traffic should now be forwarded to its network
 - E.g. what if that network has limited bandwidth?
 - => multicast senders may not implement flow control => congestion collapse
 - => if senders do implement flow control then that whole group has to slow down
- So global support is very limited
 - Selective wide-area multicast support can be provided by an **overlay network**
 - E.g. <https://en.wikipedia.org/wiki/Mbone>
to watch space shuttle launches over the Internet in the mid-1990s :-)

Multicast (& Broadcast) applications

- **Performance/network efficiency** – single physical copy can reach all machines on (local) network
 - E.g. re-imaging all the machines in A32
 - Potential benefit depends a lot on network/hardware details
- **Fault tolerance** – multiple servers can receive/handle a single multicast request
- **Service discovery** – multicast request when server IP is unknown,
 - e.g. (see later) in DHCP (broadcast), UPnP, multicast-DNS, Bonjour (multicast)
- Local **Peer-to-Peer** applications
 - E.g. (see later) LAN gaming – often discovery plus efficiency

Broadcast & Multicast Applications (including Discovery)

Note, the technical details of these applications are not examined but the general principles of discovery and the uses of multicast/broadcast are

Example: DHCP

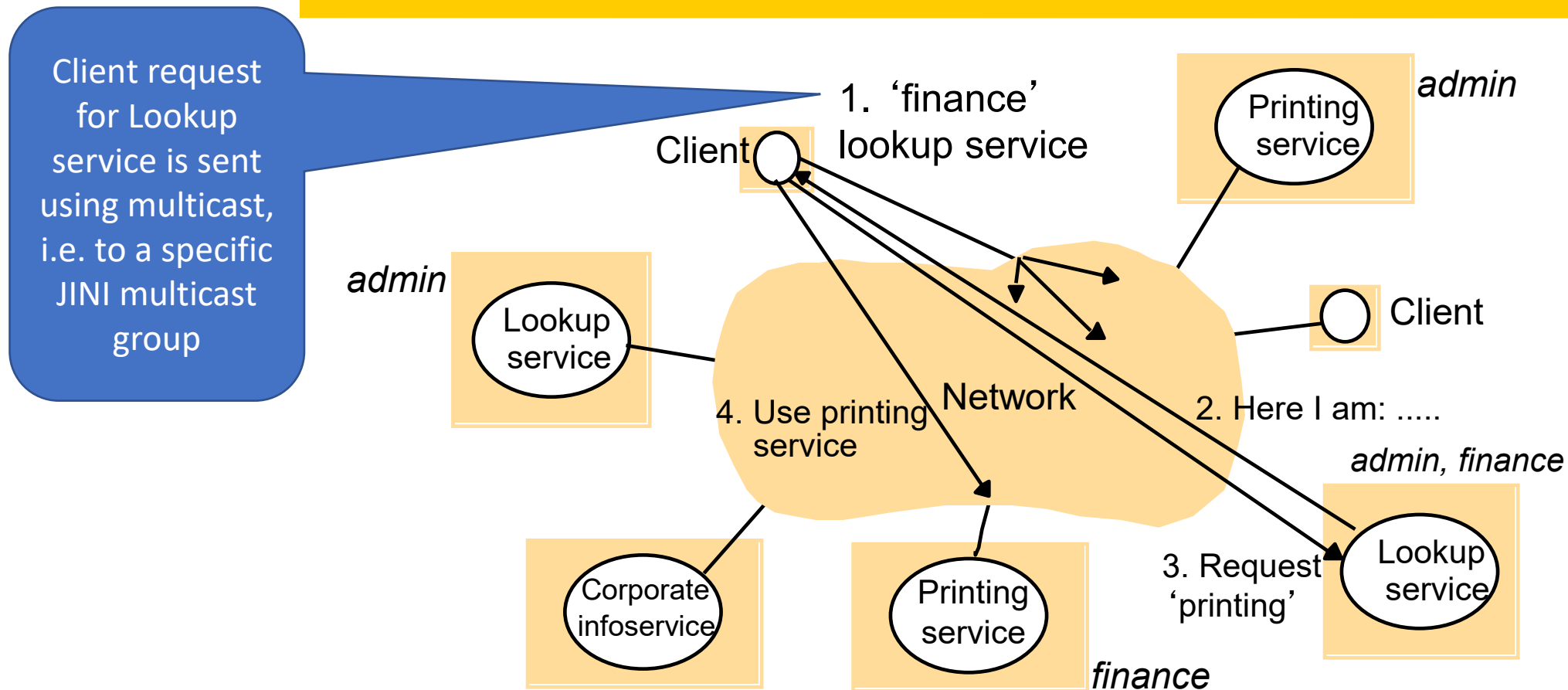
- Every host on the Internet needs to know its IP address, network, gateway, DNS server
- Often discovered when the machine connects to a local network by using **DHCP** (Dynamic Host Configuration Protocol)
 - New machine **broadcasts** a DHCP request to the local network
 - Delivered to every machine on the network
 - So new machine doesn't need to know its IP or the DHCP server's IP
 - Any DHCP server on the local network handles the request and provides the information (unicast)
 - And/or the router is configured to forward the request to a DHCP server

Example: Network Discovery services

- **Discovery** services are directory services that typically operate over local networks where devices and processes are initially unknown to one another
 - E.g. to find any printers on the local network
- Various implementation approaches
 - Often use network multicast initially
 - Either to find a discovery service, or to find other processes directly
 - E.g. Java JINI (service-based)...

Q: Have you used network discovery?

Figure 19.4 Service discovery in JINI



<https://river.apache.org/release-doc/2.2.2/specs/html/discovery-spec.html> - retired :-/

Example: **SSDP**, Simple Service Discovery Protocol

- Text-based protocol over UDP
 - https://en.wikipedia.org/wiki/Simple_Service_Discovery_Protocol
- Services are periodically **announced** by sending to (pre-defined):
 - **Multicast** group 239.255.255.250 (IPv4 site-local address)
 - UDP port 1900
- Clients can **multicast** a **search** request
- Used for service discovery in **UPnP** (Universal Plug'n'Play)
 - E.g. to find media servers, etc., on home networks

Examples: mDNS, DNS Service Discovery

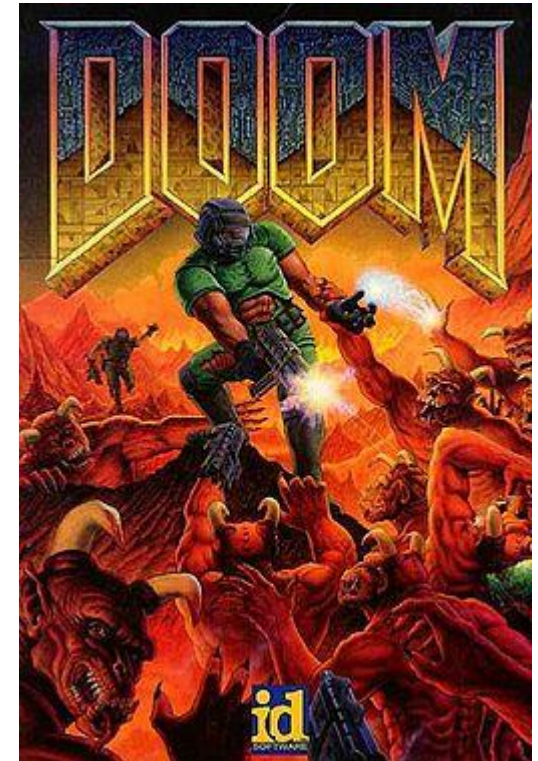
- **mDNS** = DNS over multicast
 - To look up local machine IP addresses (given their name)
 - E.g. used to look up “NAME.local” domain name
 - = machine called “NAME” on local network, which will reply
 - Supported by MacOS; support in other OSs variable
 - DNS queries are sent to multicast group 224.0.0.251, port 5353
 - All implementing hosts join the group and can respond
- **DNS Service Discovery** with mDNS
 - To find local services just from their type
 - E.g. all local printers
 - Also known as Zero-conf/Bonjour
 - (see next slide)

DNS Service Discovery process

- Client wants to find something, e.g. a web page
- Client sends **mDNS query** for **PTR** record for desired service
 - As domain “_SERVICE._PROTO.DOMAIN”, e.g. “_http._tcp.local”
- All local hosts offering that service reply
 - E.g. RR: `_http._tcp.local PTR MyWebPage._http._tcp.local`
- Client select instance of interest and queries for DNS **SRV** and **TXT** records for that domain name
 - to get service (host/port)
 - E.g. RR: `MyWebPage._http._tcp.local SRV 0 1 myhost.local 8080`
 - and additional information (e.g. name)
 - E.g. RR: `MyWebPage._http._tcp.local TXT “(name, etc., encoded)”`
- E.g. => A web page “MyWebPage” is available from host myhost.local on TCP port 8080 with name “name”...

Example: LAN gaming (DOOM)

- Original DOOM LAN multiplayer, c.1993 (apparently¹)
 - Used IPX = comparable to UDP (but by Novell, not an Internet standard)
 - On a well-known IPX network socket (c.f. UDP port)
 - Used **broadcast** to find machines and send data,
 - Used negative acknowledgements (NACKs) for reliability
 - i.e. receive a packet with a higher sequence number than expected
=> lost packet => request retransmission of missing packet
- So **all** machines received **all** data packets
 - So system admins tended to ban Doom 😊
 - Using IP Multicast would have meant that only host actually running DOOM would join the group and receive the packets (ideally) ¹https://doom.fandom.com/wiki/Doom_networking_component



Summary

- IP **Broadcast** delivers a datagram to **ALL** hosts on a physical network, e.g. IP 255.255.255.255 = “this” network
- IP **Multicast** defines a set of multicast **group** addresses
 - Any hosts can **join** or **leave** a multicast group
- Delivery to hosts/group members is **best-effort** (UDP) service
 - Note: members need to use the same UDP port number
- Efficient, but only widely supported on **local networks**
 - Esp. Ethernet and WiFi
- Useful for **service discovery**, **fault tolerance** and **efficient** data dissemination
 - E.g. DHCP (broadcast), mDNS, Bonjour, UPnP (multicast)

Recap: Multicast applications

- **Performance/network efficiency** – single physical copy can reach all machines on (local) network
 - E.g. re-imaging all the machines in A32
- **Fault tolerance** – multiple servers can receive/handle a single multicast request
- **Service discovery** – multicast request when server IP is unknown,
 - e.g. in **DHCP, multicast-DNS, Bonjour**
- Local **Peer-to-Peer** applications
 - E.g. **LAN gaming** – often discovery plus efficiency

Next

- Lab 8: IP Multicast
- Lecture 17: Distributed algorithms